

Outline

- Why learn programming?
 - What is program?
 - What is programming language?
 - What is translator program?
 - What are types of translator program?
 - Difference between compiler and interpreter.
 - Algorithm
 - Flowchart
 - Program development lifecycle
 - C background
- Why use C?
 - Features of C
 - Get Started with C
 - Install IDE
 - Basic Structure of the C Program

Why Learn Programming?

Problem-Solving Skills :

- ✓ Programming teaches you how to break down complex problems into smaller, manageable parts.
- ✓ It develops logical thinking and analytical skills that are applicable to all aspects of life.

Career Opportunities :

- ✓ Programming is a highly sought-after skill in industries like tech, finance, healthcare, and entertainment.
- ✓ Roles like software developer, data scientist, AI engineer, and cyber security analyst are in high demand.

Automation of Tasks :

- ✓ You can automate repetitive tasks, saving time and effort.
- ✓ For example: Automating data entry. Writing scripts to analyze large datasets.

Create Your Own Projects :

- ✓ Build apps, websites, games, or tools to solve real-world problems or for fun.
- ✓ Turn your ideas into reality, whether it's a personal blog or a revolutionary app.

Why Learn Programming? Cont..

High Income Potential :

- ✓ Programmers are among the highest-paid professionals worldwide.
- ✓ Freelancing opportunities allow you to earn from anywhere in the world.

Future-Proof Skill

- ✓ As technology advances, programming will continue to be a foundational skill in the workforce.
- ✓ Learning programming prepares you for a world increasingly driven by technology.

Better Understanding of Technology :

- ✓ Programming gives you insight into how software, websites, and systems work.
- ✓ It empowers you to make informed decisions in a tech-driven world.

What is program?

- A *program* is a set of instructions that a computer uses to perform a specific function.
- To use an analogy, a program is like a computer's recipe.
- It contains a list of ingredients (called variables, which can represent numeric data, text, or images) and a list of directions (called statements) that tell the computer how to execute a specific task.

What is programming language?

- Programs are created using specific programming languages such as C++, Python, and Ruby.
- These are high level programming languages that are human-readable and writable.
- These languages are then translated into low level machine languages by compilers, interpreters, and assemblers within the computer system.
- Assembly language is a type of low level language that is one step above a machine language and technically *can* be written by a human, although it is usually much more cryptic and difficult to understand.

Translator Program

- A translator is a programming language processor that converts a computer program from one language to another.
- It takes a program written in source code and converts it into machine code.
- It discovers and identifies the error during translation.

Types of translator program

Three types of translator :

- Compiler
- Interpreter
- Assembler

Compiler

- Compiler is a translator which is used to convert programs in high-level language to low-level language.
- It translates the entire program and also reports the errors in source program encountered during the translation.
- The compiler specifies the errors at the end of compilation, if there are any errors in the source code.
- The error must be removed before the compiler can successfully recompile the source code again.

Example :

- GCC->GNU compiler collection
- COBOL->Common Business Oriented Language
- Language that use compiler :
- C, C++, GO, Swift etc..

Interpreter and Assembler

Interpreter

- Interpreter is a translator which is used to convert programs in high-level language to low-level language.
- Interpreter translates line by line and reports the error once it encountered during the translation process.
- Language : PHP, Python, Ruby, Javascript

Assembler

- Assembler is a translator which is used to translate the assembly language code into machine language code.
- The source program written in the assembly language is given as the input to the assembler.
- The output generated by the assembler is the object code or machine code understandable by the computer.
- Example :
 - FAP->Fortran Assembly Program
 - MAP->Macro Assembly Program

Difference between Compiler and Interpreter

S.No	Compiler	Interpreter
1.	Compiler scans the whole program in one go.	Translates program one statement at a time.
2.	As it scans the code in one go, the errors (if any) are shown at the end together.	Considering it scans code one line at a time, errors are shown line by line.
3.	Main advantage of compilers is it's execution time.	Due to interpreters being slow in executing the object code, it is preferred less.
4.	It converts the source code into object code.	It does not convert source code into object code instead it scans it line by line
5	It does not require source code for later execution.	It requires source code for later execution.
Eg.	C, C++, C# etc.	Python, Ruby, Perl, SNOBOL, MATLAB, etc.

Algorithm

- In computer programming terms, an algorithm is a set of well-defined instructions to solve a particular problem.
- It takes a set of input and produces a desired output. For example,
- An algorithm to add two numbers:
 1. Take two number inputs
 2. Add numbers using the + operator
 3. Display the result

Characteristics/Qualities of Good Algorithms

- Input and output should be defined precisely.
- Each step in the algorithm should be clear and unambiguous.
- Algorithms should be most effective among many different ways to solve a problem.
- An algorithm shouldn't include computer code. Instead, the algorithm should be written in such a way that it can be used in different programming languages.

Example - Algorithm

Algorithm 1: Add two numbers entered by the user

```
Step 1: Start
Step 2: Declare variables num1, num2 and sum.
Step 3: Read values num1 and num2.
Step 4: Add num1 and num2 and assign the result to sum.
        sum←num1+num2
Step 5: Display sum
Step 6: Stop
```


Flowchart

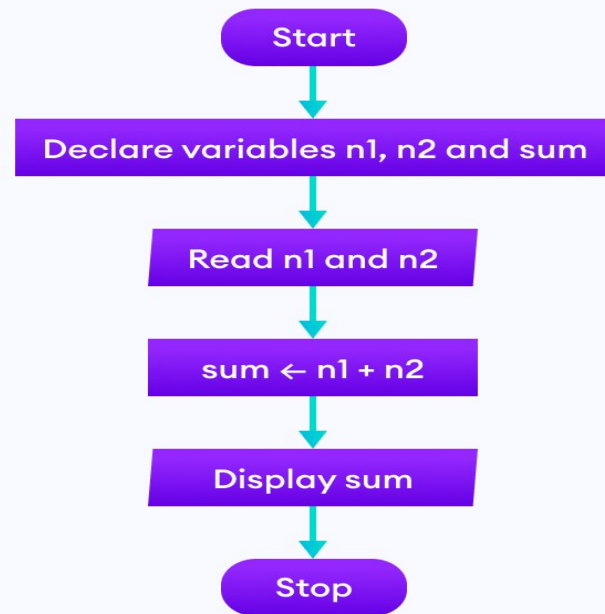
- A flowchart is a diagrammatic representation of an algorithm.
- A flowchart can be helpful for both writing programs and explaining the program to others.

Flowchart

Symbol	Purpose	Description
	Flow line	Indicates the flow of logic by connecting symbols.
	Terminal(Stop/Start)	Represents the start and the end of a flowchart.
	Input/Output	Used for input and output operation.
	Processing	Used for arithmetic operations and data-manipulations.
	Decision	Used for decision making between two or more alternatives.
	On-page Connector	Used to join different flowline

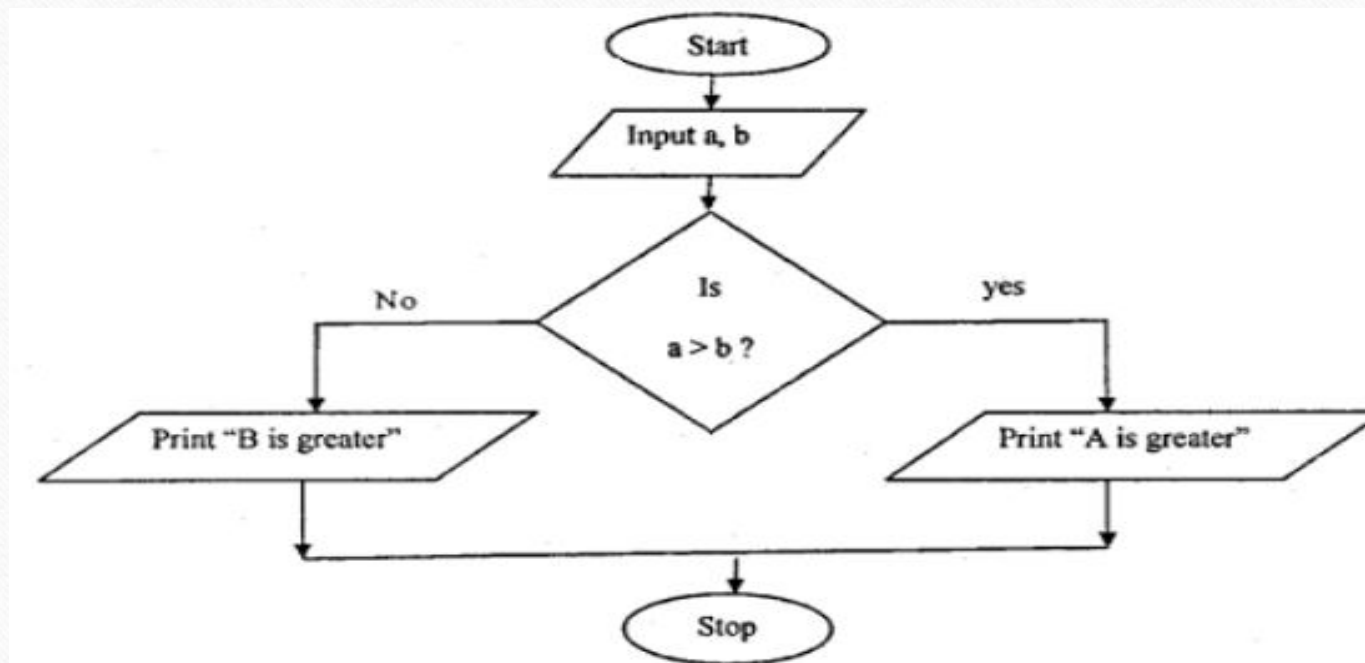
Flowchart Example

Flowchart to add two numbers :



Exercise : Draw a flowchart to add three numbers.

Flowchart to Find Largest from Two Numbers



Difference between algorithm and flowchart

Algorithm	Flowchart
Algorithm is step by step procedure to solve the problem.	Flowchart is a diagram created by different shapes to show the flow of data.
Algorithm is complex to understand.	Flowchart is easy to understand.
In algorithm plain text are used.	In flowchart, symbols/shapes are used.
Algorithm is easy to debug.	Flowchart it is hard to debug.
Algorithm is difficult to construct.	Flowchart is simple to construct.
Algorithm does not follow any rules.	Flowchart follows rules to be constructed.
Algorithm is the pseudo code for the program.	Flowchart is just graphical representation of that logic.

Program development lifecycle

- When we want to develop a program using any programming language, we follow a sequence of steps. These steps are called phases in program development.
- The program development life cycle is a set of steps or phases that are used to develop a program in any programming language.
- Generally, the program development life cycle contains 6 phases, they are as follows....
 - Problem Definition
 - Problem Analysis
 - Algorithm Development
 - Coding & Documentation
 - Testing & Debugging
 - Maintenance

Program development lifecycle cont..

Problem Definition :

- In this phase, we define the problem statement and we decide the boundaries of the problem.
- In this phase we need to understand the problem statement, what is our requirement, what should be the output of the problem solution.
- These are defined in this first phase of the program development life cycle.

Problem Analysis :

- In phase 2, we determine the requirements like variables, functions, etc. to solve the problem.
- That means we gather the required resources to solve the problem defined in the problem definition phase.
- We also determine the bounds of the solution.

Program development lifecycle cont..

Algorithm Development :

- During this phase, we develop a step by step procedure to solve the problem using the specification given in the previous phase.
- This phase is very important for program development.
- That means we write the solution in step by step statements.

Coding & Documentation :

- This phase uses a programming language to write or implement the actual programming instructions for the steps defined in the previous phase.
- In this phase, we construct the actual program.
- That means we write the program to solve the given problem using programming languages like C, C++, Java, etc.

Program development lifecycle cont..

Testing & Debugging :

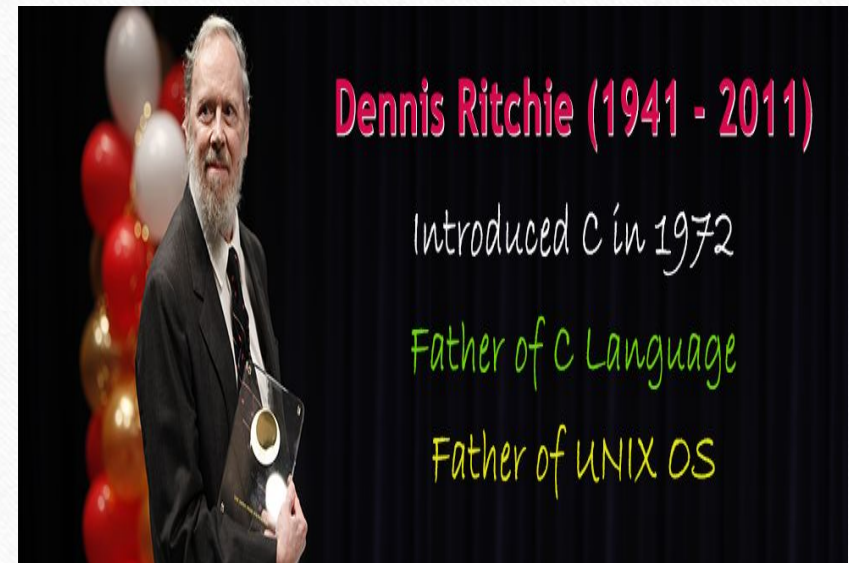
- During this phase, we check whether the code written in the previous step is solving the specified problem or not.
- That means we test the program whether it is solving the problem for various input data values or not.
- We also test whether it is providing the desired output or not.

Maintenance :

- During this phase, the program is actively used by the users. If any enhancements found in this phase, all the phases are to be repeated to make the enhancements.
- That means in this phase, the solution (program) is used by the end-user. If the user encounters any problem or wants any enhancement, then we need to repeat all the phases from the starting, so that the encountered problem is solved or enhancement is added.

C background

- C is a structured programming language. It is also known as function orientated programming language.
- C programming language was developed in the year of 1972 by **Dennis Ritchie** at Bell Laboratories in the USA (AT & T)
- .In the year of 1968, research was started by Dennis Ritchie on programming languages like BCPL, CPL.
- The main aim of his research was to develop a new language to create an OS called UNIX.
- After four years of research, a new programming language was created with solutions for drawbacks in languages like BCPL & CPL.
- In the year of 1972, the new language was introduced with the name “**Traditional C**”.



Why Use C?

- C was initially used for system development work, particularly the programs that make-up the operating system.
- C was adopted as a system development language because it produces code that runs nearly as fast as the code written in assembly language.
- Some examples of the use of C might be –
 - Operating Systems
 - Language Compilers
 - Assemblers
 - Text Editors
 - Print Spoolers
 - Network Drivers
 - Modern Programs
 - Databases
 - Language Interpreters
 - Utilities

Features of C

Simple :

- C is a simple language in the sense that it provides a **structured approach** (to break the problem into parts), **the rich set of library functions, data types**, etc.

Machine Independent or Portable :

- C is a simple language in the sense that it provides a **structured approach** (to break the problem into parts), **the rich set of library functions, data types**, etc.

Structured programming language :

- C is a structured programming language in the sense that **we can break the program into parts using functions**.
- So, it is easy to understand and modify. Functions also provide code reusability.

Rich Library :

- C **provides a lot of inbuilt functions** that make the development fast.

Memory Management :

- It supports the feature of **dynamic memory allocation**.
- In C language, we can free the allocated memory at any time by calling the **free()** function.

Features of C cont..

Speed

- The compilation and execution time of C language is fast since there are lesser inbuilt functions and hence the lesser overhead.

Pointer

- C provides the feature of pointers. We can directly interact with the memory by using the pointers. We **can use pointers for memory, structures, functions, array**, etc.

Recursion

- In C, we **can call the function within the function**. It provides code reusability for every function. Recursion enables us to use the approach of backtracking.

Extensible

- C language is extensible because it **can easily adopt new features**.

Get Started

To start using C, you need two things:

- A text editor, like Notepad, to write C code
- A compiler, like GCC, to translate the C code into a language that the computer will understand
- There are many text editors and compilers to choose from.

Install IDE

- An IDE (Integrated Development Environment) is used to edit AND compile the code.
- Popular IDE's include Code::Blocks, Eclipse, and Visual Studio.
- These are all free, and they can be used to both edit and debug C code.
- **Note:** Web-based IDE's can work as well, but functionality is limited.
- We will use **Code::Blocks**

Basic Structure of the C Program

- Documentation
- Link
- Definition
- Global Declaration
- Main () Function
- Subprograms

Basic Structure of the C Program cont..

Documentation :

- Consists of the description of the program, programmer's name, and creation date.
- These are generally written in the form of comments.

Link :

- All header files are included in this section which contains different functions from the libraries.
- A copy of these header files is inserted into your code before compilation.

Definition :

- Includes preprocessor directive, which contains symbolic constants.
- E.g.: #define allows us to use constants in our code. It replaces all the constants with its value in the code.

Basic Structure of the C Program cont..

Global Declaration :

- Includes declaration of global variables, function declarations, static global variables, and functions.

Main Function :

- For every C program, the execution starts from the main() function.
- It is mandatory to include a main() function in every C program.

Subprograms :

- Includes all user-defined functions (functions the user provides).
- They can contain the inbuilt functions and the function definitions declared in the Global Declaration section.
- These are called in the main() function.

Sample C Program

```
/**                                     //Documentation
 * file: age.c
 * author: you
 * description: program to find our age.
 */

#include <stdio.h>                     //Link

#define BORN 2000                      //Definition

int age(int current);                 //Global Declaration

int main(void)                        //Main() Function
{
    int current = 2021;
    printf("Age: %d", age(current));
    return 0;
}

int age(int current) {                //Subprograms
    return current - BORN;
}
```

Thank You

